AI That Decides

Jev, System One Models, and the Architecture of Fast Decisions

Matteo Abis

An engineering walkthrough · 40 minutes + 5 minutes discussion

Source snapshot: 23 October 2026 · TypeSafe examples pin Jev 1.13.0

Speaker notes

I’m Matteo Abis. Today we will look at what changes when software can ask a model for a decision directly, instead of asking for text and interpreting the answer.

We will start with a transaction, build the basic idea from familiar LLM components, then look at calibration and an open implementation. We will finish with benchmarks and two practical workflows. The question to keep in mind is: where do we need intelligence, and where do we simply need the answer in a useful form?

Sources (checked 2026-10-23): TypeSafe · System One

Meet Jev

Give it evidence and typed questions.
Get decision probabilities directly.

One shared state. Three question types.

Choice Which category?
Probabilities over your named options.

Score Where on a scale?
Ordered levels, probabilities and a score.

Noul Yes or no?
A number from 0 to 1: P(yes).

API price · USD / 1M tokens

Model	Input	Output
Jev 1.13	**$0.042**	**Free**
GPT-6 Luna	$0.10	$0.50
GPT-6 Sol	$2.00	$10.00
GPT-6 Astra	$10.00	$50.00

23 Oct 2026 · Standard, uncached input; OpenAI short-context (≤272K). No regional surcharges.

Typesafe · model pricing · OpenAI · API pricing · Typesafe · API · Typesafe · launch, 15 Sep 2026

Speaker notes

Jev is TypeSafe's decision model, publicly released in early access on 15 September 2026. Send it shared evidence and typed questions. It returns probabilities directly.

The three interfaces are Choice, Score and Noul. We will see examples next. Notice the pricing shape: Jev charges for input tokens and has no output-token charge. The comparison here is a dated API price snapshot, not a cost-per-correct-decision benchmark. Actual workflow cost also depends on prompt length, repeated calls and how many cases need further work.

Sources (checked 2026-10-23): TypeSafe · model pricing · OpenAI · API pricing · TypeSafe · API · TypeSafe · launch, 15 Sep 2026

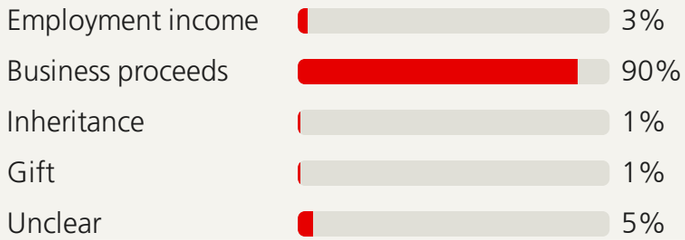
Three Ways to Ask

Choice

Pick a category

What is the stated source of funds?

"Proceeds from my consultancy business."

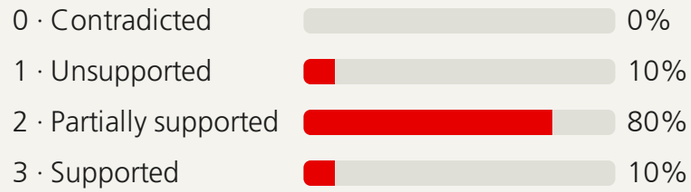


Score

Rate on ordered levels

How strongly do the documents support the explanation?

Invoice matches the amount; payment trail is incomplete.



Expected score: 2.0 / 3

Noul

Judge a proposition

Does the passage explicitly identify the client as seller?

"The seller, Mr Example, transfers ownership to the buyer."

0.97 P(yes)

97% probability of "yes".

Your code chooses the threshold.

TypeSafe · Choice · TypeSafe · Score · TypeSafe · Noul

Speaker notes

Choice asks which named category fits. Score asks where something belongs on an ordered scale. Noul asks whether a proposition is true and returns a probability of yes.

In the middle example, an incomplete payment trail means partially supported; missing evidence is not automatically contradiction. Score preserves the order of the levels, while Choice treats its labels as separate categories.

Do not look only at the expected score. A fifty-fifty split between 0 and 3 has the same mean as a fifty-fifty split between 1 and 2, but tells a very different story. Keep the distribution when uncertainty matters.

Sources (checked 2026-10-23): TypeSafe · Choice · TypeSafe · Score · TypeSafe · Noul

What Changes When the Answer Is a Decision?

Let’s start with a conventional LLM.

Speaker notes

We have seen the interface. Now let’s understand why it changes the computation.

Start with the familiar approach: give a conventional LLM the evidence, then ask it to return JSON.

Sources (checked 2026-10-23): Presenter’s conceptual explanation or illustrative example; not measured model output.

A Transaction Arrives. Classify It.

ACCOUNT ACTIVITY · 20 OCT 2026

+ USD 1,000,000.00

Mr Example ← Test Corp

Dividend credit · Ref. DIV-2026-1042

TEST CORP — DIVIDEND STATEMENT

Registered holder	Mr Example
Dividend entitlement	100,000 shares × USD 10.00

Net dividend paid: USD 1,000,000.00

Paid on 20 October 2026. Deductions: USD 0.00.
Payment reference: DIV-2026-1042.

Corporate Secretary · Test Corp

CONVENTIONAL LLM

Classify the source of wealth.

Return a category and confidence as JSON.

Complete

```
{
  "source_of_wealth": "business",
  "confidence": "high"
}
```

Speaker notes

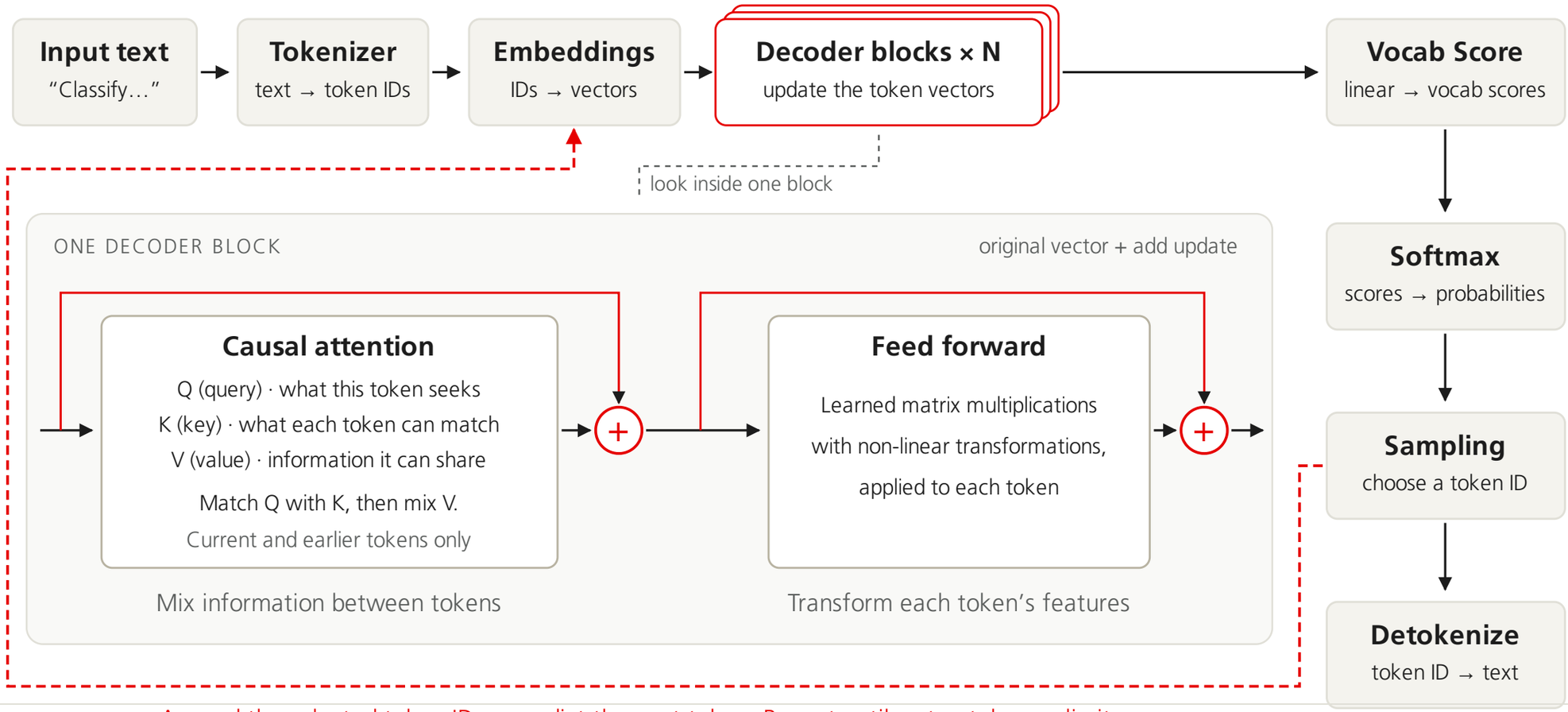
Mr Example receives a million-dollar payment from Test Corp. The supporting statement records one hundred thousand shares and a ten-dollar dividend per share. The amount and payment reference match.

On the right, watch the LLM produce the JSON one piece at a time. Use Replay if needed. Our software wants a category, but the model also emits braces, field names, quotation marks and a confidence word.

“High” is a generated label. It is not automatically a measured probability of being correct. To see where the time goes, let’s follow the computation inside the model.

Sources (checked 2026-10-23): Presenter’s conceptual explanation or illustrative example; not measured model output.

Inside a language model



Simplified text-generation path through a modern decoder.

Llama 2 · decoder design · The Illustrated GPT-2 · Transformer Explainer

Speaker notes

Follow the top row: text becomes token IDs, then vectors. Repeated decoder blocks update those vectors. The vocabulary projection assigns scores to possible next tokens. Softmax turns scores into probabilities; sampling selects a token.

Inside a block, attention mixes information between tokens. Queries describe what to look for, keys provide matching information, and values carry the content to combine. Causal attention sees the current and earlier positions. Feed forward transforms each token's features through learned matrix operations and nonlinear functions. The red paths add each update back to the original vector.

Finally, the selected token goes back in. That loop generates the next token.

Sources (checked 2026-10-23): Llama 2 · decoder design · The Illustrated GPT-2 · Transformer Explainer

Behind the JSON: prefill and decode

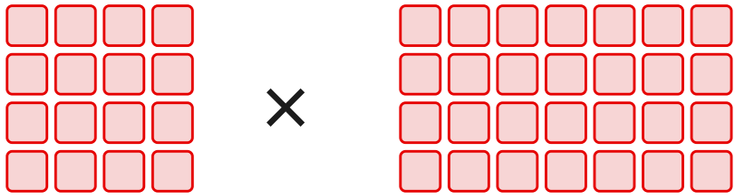
1 / Prefill

~2,000 tokens/s*

The whole prompt goes in at once.

Instructions | Transaction | Document

Model weights × All prompt vectors



matrix × matrix

Each column is one token vector.

Full-prompt pass → first output token

Matrix × matrix operations across the model.

2 / Decode

~50 tokens/s*

One token vector at a time, through the same model.

Model weights × One token vector



Embed token → The same model (one token vector at a time) → Vocab Score

next token → next forward pass

KV CACHE — Prompt + 13 generated tokens

Prompt K, V



Complete

```
{
  "source_of_wealth": "business",
  "confidence": "high"
}
```

NVIDIA · prefill & decode · Hugging Face · KV caching · AutoTrust · backbone config

*Indicative throughput, not a benchmark. Depends on hardware, context and batching. Animation slowed for teaching.

Speaker notes

On the left, all prompt tokens are already known. The model processes their positions together, layer by layer. This creates large matrix-to-matrix operations that use accelerators efficiently. The final prompt position already supplies the scores for the first output token.

On the right, the next input depends on the previous output. For one sequence, decoding advances one token vector at a time. The KV cache keeps earlier attention information, so it need not all be recomputed, but each new token still passes through the model.

The throughput figures illustrate the contrast, not a universal ratio. Ask: which computation could disappear if we only wanted a category?

Sources (checked 2026-10-23): NVIDIA · prefill & decode · Hugging Face · KV caching · AutoTrust · backbone config

Do we need to spell it out?

Prefill has already processed the whole prompt.

We only need a category.

Could we read the decision directly?

Speaker notes

Pause here. The model has already processed all the evidence and produced a next-token distribution. For a simple bounded decision, that may be enough.

This does not mean every problem is solved during prefill: additional reasoning can improve difficult answers. Our narrower question is whether we need an entire generated sentence or JSON object when we only want one choice.

Sources (checked 2026-10-23): Presenter’s conceptual explanation or illustrative example; not measured model output.

Vocab Score: One Number per Token

The vocabulary is the model's inventory of token IDs:
whole words, word parts, punctuation and special tokens.

Qwen 3.8–27B: 248,320 vocabulary entries. Each raw score is a logit.

the	bank	receives	dividend	ing	,	{	"	…
1.2	–0.4	2.1	3.8	–1.6	0.7	2.5	1.9	…

AutoTrust · backbone config

Speaker notes

The vocabulary is the inventory of token IDs the model can use. Tokens can represent whole words, parts of words, punctuation or special symbols. The Qwen configuration used here has 248,320 entries.

After the full decoder stack, the final token's hidden vector is projected into one number for every vocabulary entry. These numbers are logits: raw scores, not probabilities. Higher means more preferred at this position.

The scores shown here are just examples. The useful observation is that the model already scores possible answers before it samples anything.

Sources (checked 2026-10-23): AutoTrust · backbone config

Let’s Build Jev at Home

Mr Example receives USD 1M from Test Corp.
The dividend statement confirms the payment.

Classify the source of funds.

A Business B Inheritance C Employment D Gift

What is the right choice: A, B, C or D?
Reply with only the letter.

`Answer:` |

Use labels that each map to one token at the answer position.

Speaker notes

Keep the transaction and statement, but change the requested answer. Map Business to A, Inheritance to B, Employment to C and Gift to D. Ask for only the letter, then stop at the answer position.

Choose labels that are each one token in that exact context. Whitespace and tokenization matter. The meaning of A comes from the prompt; it is not permanently tied to Business.

We now have four relevant vocabulary entries. Instead of sampling one and continuing generation, we can read those entries directly.

Sources (checked 2026-10-23): SemIf · readout

Read the Probabilities

After softmax over the full vocabulary:

the	{	"	A	B	C	D	rest
0.10	0.20	0.15	0.34	0.02	0.02	0.02	0.15

Keep A, B, C and D. Divide each by their total.

$$q_i = \frac{p_i}{p_A + p_B + p_C + p_D} \qquad q_A = \frac{0.34}{0.40} = 85\%$$

$$0 \le q_i \le 1 \qquad \sum_{i \in \{A,B,C,D\}} q_i = 1$$

Business 85% · Inheritance 5% · Employment 5% · Gift 5%

Speaker notes

Point to A, B, C and D in red. Their probabilities under the full vocabulary add up to 0.40. Divide each by that total to get a distribution over our four choices.

For A, 0.34 divided by 0.40 gives 0.85. The other three become 0.05 each. Every result lies between zero and one, and the four sum to one. Applying softmax directly to only those four logits gives the same result.

We have removed sampling and the answer-generation loop. But normalization alone does not make these numbers reliable probabilities of correctness. We will address that shortly.

Sources (checked 2026-10-23): Semlf · readout

Jev 101: Summary

The machinery is already there

An LLM already turns context into token scores.

A different readout turns it into a decision engine.

Skip the expensive part

Read the decision after one prompt pass.

Eliminate the answer-generation loop to save time and compute.

Bring the world knowledge with you

Pretraining supplies language, concepts and relationships.

That knowledge transfers to new examples.

New decisions, no retraining

Describe the question and choices in the prompt.

The same model can tackle a new classification task.

Semlf · readout · Brown et al. · learning from prompts

Speaker notes

Four ideas so far. The scoring machinery already exists in an LLM. Reading a decision directly removes the generated-answer loop. The pretrained model brings broad language and world knowledge. And the prompt can define new questions and categories without a separate training run for each task.

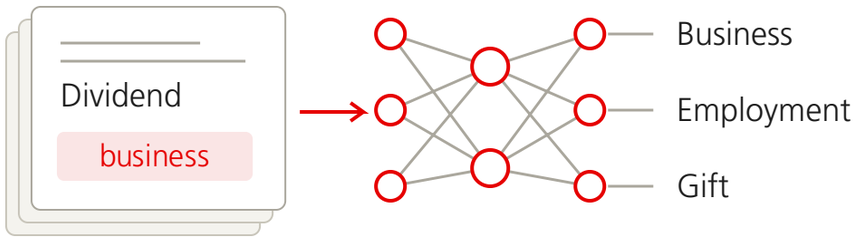
This is the basic idea, not a reconstruction of TypeSafe's private implementation. It explains why the approach is attractive. The next questions are how far that knowledge transfers, how several questions can share work, and how to make the probabilities useful.

Sources (checked 2026-10-23): SemIf · readout · Brown et al. · learning from prompts

“Just a Classifier”?

Train a model for a task

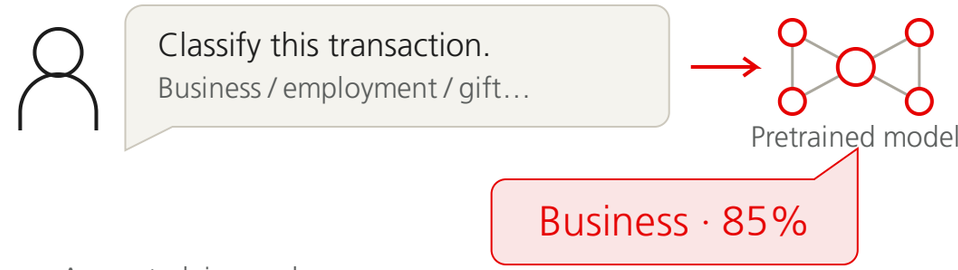
Collect labeled examples.
Learn the task’s categories.



Examples + labels Train the weights

Describe a task to a model

Start with broad language and world knowledge.
Supply the question and choices.



A new task in words

Transfer what it knows to a task you just described.

> “Jev’s impressive breakthrough is that it generalizes so well”
>
> Sebastian Raschka · 20 September 2026

Sebastian Raschka · Jev & generalization · Brown et al. · learning from prompts

Speaker notes

Classification itself is old. The interesting change is what the model already knows before we describe the task.

On the left, the familiar workflow collects labeled examples and trains a task-specific model. On the right, a pretrained language model can use a task described in words, drawing on knowledge learned across many domains. That makes new questions much easier to try.

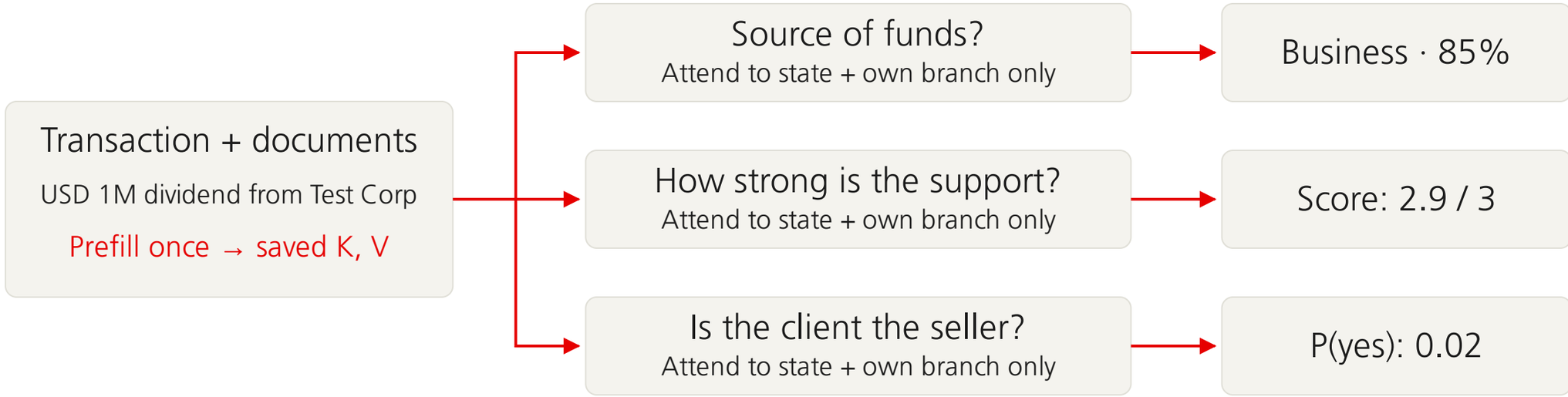
Earlier classifiers could also use transfer learning. The point is the breadth and flexibility of this interface, not that transfer is new. Nor does it guarantee performance in an unseen domain. That still needs evaluation.

Sources (checked 2026-10-23): Sebastian Raschka · Jev & generalization · Brown et al. · learning from prompts

Reuse the Work. Isolate the Questions.

Put the transaction and supporting documents before the questions.

Causal attention cannot look ahead: later questions cannot change its cached vectors.



Isolation: the attention mask hides other branches.

Efficiency: reuse the state cache; batch the question branches together.

Hugging Face · KV caching · Semlf · readout

Speaker notes

Use the same transaction for three questions: source of funds, strength of evidence, and whether the client is the seller.

The long state comes first. In a causal model, later questions cannot change the earlier state representations. We can compute the state once and reuse its cached keys and values. Each question branch can attend to that state and its own tokens, while an attention mask hides the other branches.

This gives shared work without one answer influencing another. The diagram explains an implementable branching design; it does not reveal TypeSafe's unpublished internals. The questions must be answerable from the shared evidence independently.

Sources (checked 2026-10-23): Hugging Face · KV caching · SemIf · readout

Another Question Is Only Another Branch.

10,000 tokens of transaction documents. Three 100-token questions.

Three separate requests

Read 10,000	+ 100
Read 10,000	+ 100
Read 10,000	+ 100

3 × (10,000 + 100)

30,300 input tokens

One request, three branches

Read 10,000 once	+ 100
	+ 100
	+ 100

10,000 + 3 × 100

10,300 input tokens

Ask source of funds and “Is the client the seller?” together.

The seller question can be answered directly from the documents.

If the transaction is employment income, discard the seller answer.

Asking upfront saves another call when the transaction is a sale.

TypeSafe · parallel questions · TypeSafe · speculative fan-out

Speaker notes

Walk through the token arithmetic. Three separate requests repeat the ten-thousand-token state three times. One request with three questions reads it once. That is 30,300 versus 10,300 input tokens in this simplified example.

This is why TypeSafe recommends asking even some speculative questions. We can ask whether the client is the seller at the same time as asking the source of funds. If the transaction is employment income, code discards the irrelevant seller answer.

We save a second round trip when that answer is needed. This works when the question can be answered from the documents, without waiting for another answer.

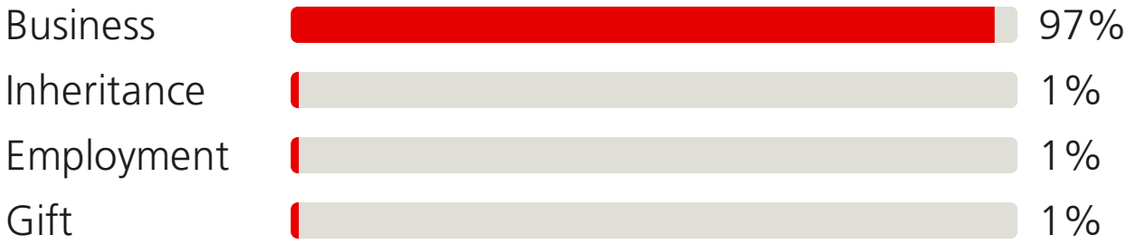
Sources (checked 2026-10-23): TypeSafe · parallel questions · TypeSafe · speculative fan-out

Adds Up to 1. Does It Deserve 97%?

Reading label-token probabilities is an old idea.

A preference for the next token is not yet a reliable estimate of correctness.

Our transaction → four options



97%

Across 100 comparable predictions at 97%, are about 97 correct?

That is the calibration question.

~~Raw readouts can be too sharp, or favor particular labels and positions.~~

Test the probabilities against outcomes—not just whether they sum to 1.

Zhao et al. · Calibrate Before Use, 2021 · Guo et al. · ICML 2017

Speaker notes

Our homemade decision model returns 97% for Business. The numbers sum to one, but does 97% mean what we want it to mean?

Ask the room: would you act on an 80% prediction? The answer depends on the consequence of an error and whether similar predictions really succeed about eight times in ten.

Language models can prefer particular labels, answer positions or familiar phrasing. Their next-token probabilities can also be too concentrated. Calibration compares predicted probabilities with observed outcomes. Accuracy asks whether the winner is right; calibration asks whether the assigned uncertainty is meaningful.

Sources (checked 2026-10-23): Zhao et al. · Calibrate Before Use, 2021 · Guo et al. · ICML 2017

Two Knobs: Bias and Temperature

$$p_i = \frac{\exp((z_i + b_i)/T)}{\sum_j \exp((z_j + b_j)/T)}$$

Bias · move individual scores

A separate offset for each option.
Correct a persistent preference for A, for example.

A: 4	+ (–1)	3

Can change which option wins.
The same offset for every option cancels out.

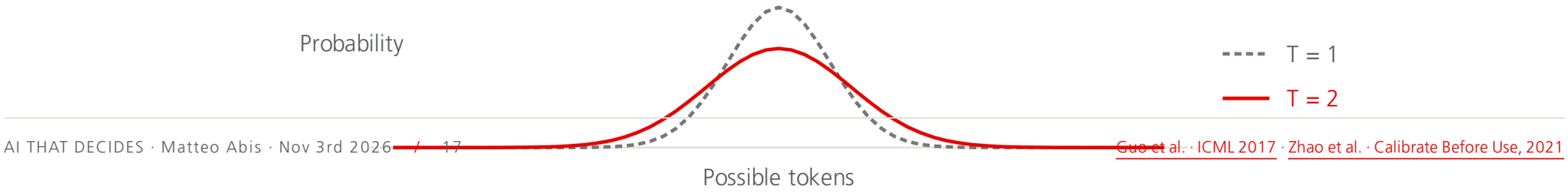
Temperature · change the spread

Divide every adjusted score by T.
T > 1 softens; 0 < T < 1 sharpens.

Same four option scores

T = 1 94% · 2% · 2% · 2%
T = 2 70% · 10% · 10% · 10%

Keeps the winning option unchanged.



Guo et al. · ICML 2017 · Zhao et al. · Calibrate Before Use, 2021

Speaker notes

There are two simple adjustments. Bias adds a separate offset to each score. It can correct a persistent preference for a label and can change which option wins. Adding the same offset everywhere changes nothing after softmax.

Temperature divides all adjusted scores by a positive number. A temperature above one spreads probability more evenly; below one makes it sharper. Point to the grey and red curves. The winner stays the same, but the probabilities change.

Fit these values on held-out examples with known outcomes. Smoothing is not automatically improvement: it helps only when the original distribution is too confident in the relevant setting.

Sources (checked 2026-10-23): Guo et al. · ICML 2017 · Zhao et al. · Calibrate Before Use, 2021

Open Implementations

SemIf · temperature alone

Keep the LLM frozen.
Fit T by minimizing label log loss.

WANLI · 256 examples

T ≈ 2.50

0.208 → 0.069

Expected calibration error, raw → out-of-fold.
Five folds; accuracy stays 63.7%.
The release temperature is fitted on all 256 rows.

AutoTrust JEV-27B · bias + T

Add the trained decision head's bias.
Apply a fitted temperature for each question type.

Noul	Choice	Score
1.014	1.016	1.004

10,954 temperature-fit rows.
Bias is learned with the head; temperature is fitted afterward.

AutoTrust's temperatures barely move the distribution.

Its adapters and decision head have already learned from soft targets.

SemIf · calibration experiment · AutoTrust · pinned implementation · AutoTrust · fitted temperatures

Speaker notes

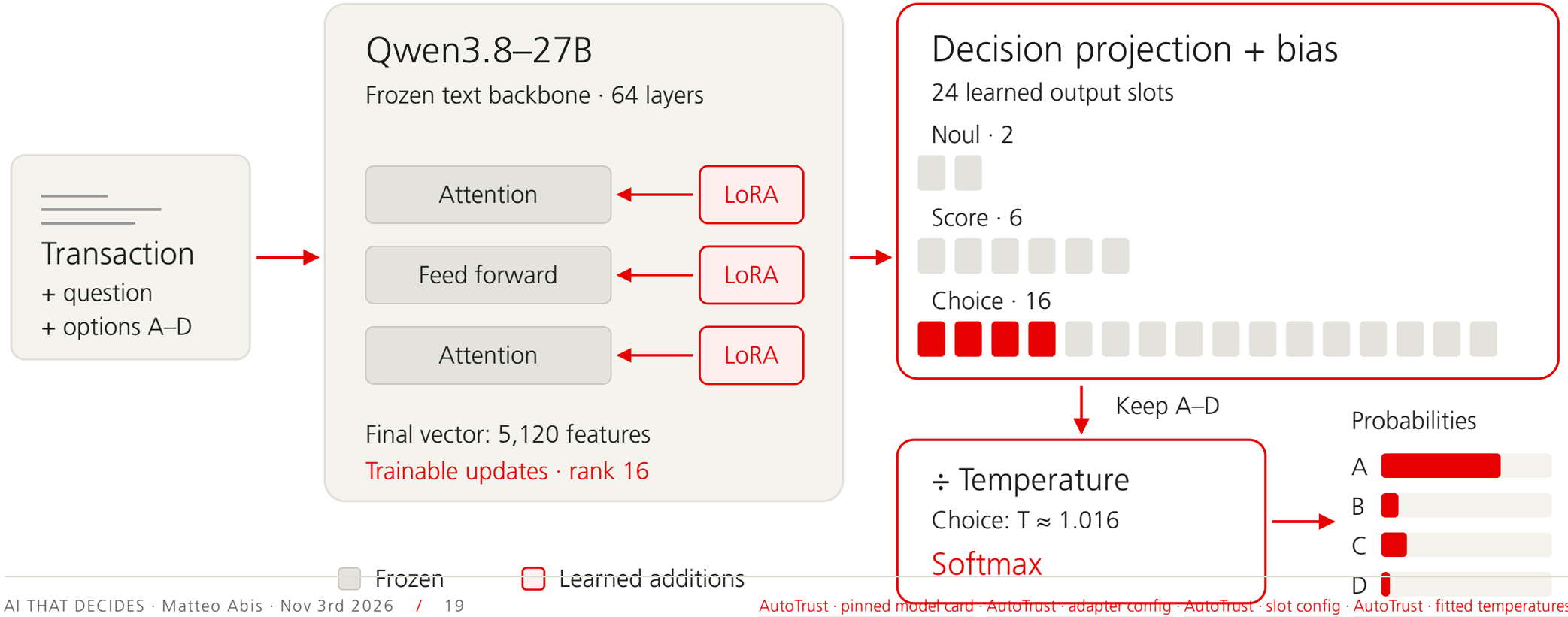
Here are two concrete implementations. SemIf keeps the model frozen and fits temperature. In its small WANLI experiment, the out-of-fold calibration error falls while accuracy stays unchanged. That is exactly what temperature alone should be able to do: adjust certainty without changing the ranking.

AutoTrust has a learned decision-head bias and a fitted temperature for each question type. Notice how close its temperatures are to one. Most of the adaptation has already happened during training of its adapters and decision head.

These are separate projects with different training and evaluation setups. Let’s open up AutoTrust’s architecture next.

Sources (checked 2026-10-23): SemIf · calibration experiment · AutoTrust · pinned implementation · AutoTrust · fitted temperatures

Inside AutoTrust JEV-27B



AutoTrust · pinned model card · AutoTrust · adapter config · AutoTrust · slot config · AutoTrust · fitted temperatures

Speaker notes

Follow the decision path from left to right. The evidence, question and options enter a frozen Qwen3.8 27B backbone. Small LoRA updates are inserted inside attention and feed-forward projections.

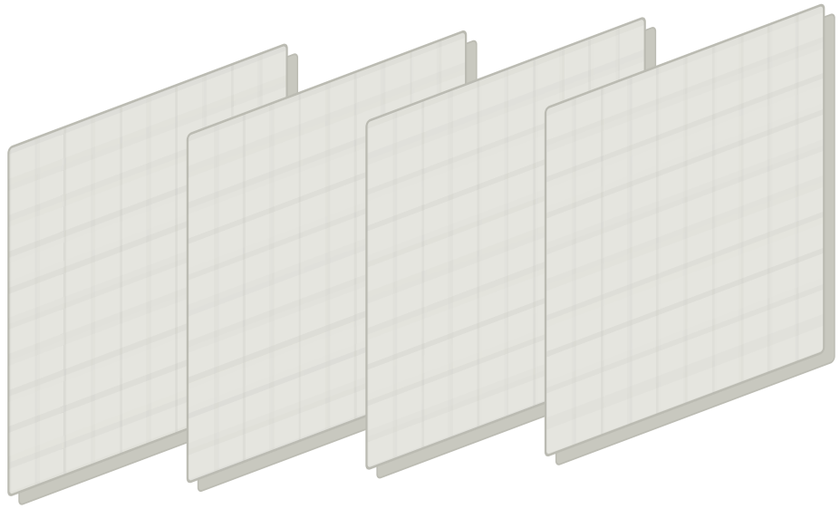
The final hidden vector feeds a trained decision projection with a learned bias. It produces 24 slots: two for Noul, six for Score and sixteen for Choice. For our four-option example, we keep four Choice slots, apply the fitted temperature and normalize.

AutoTrust is an independent open model, not TypeSafe Jev. It also retains a generation path. Here we are following its specialized decision path, whose published implementation we can inspect.

Sources (checked 2026-10-23): AutoTrust · pinned model card · AutoTrust · adapter config · AutoTrust · slot config · AutoTrust · fitted temperatures

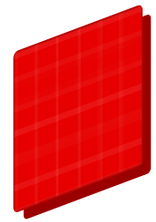
LoRA: Low Rank Adapters

Keep the original model weights frozen.
Train small updates that teach it how to make decisions.



Original weights · frozen

+



Small learned updates

108M

LoRA parameters

≈ 0.4% of the backbone

“Low rank”: two small matrices represent each weight update.
AutoTrust uses rank 16, reducing how many parameters need training.

The updates sit inside attention and feed forward, adapting how the model reads the evidence.

Hu et al · LoRA · 2021 · AutoTrust · adapter config · AutoTrust · pinned model card

Speaker notes

The grey squares represent the original weight matrices. They stay frozen. The red square represents small learned updates spread through the model, not a single layer attached at the end.

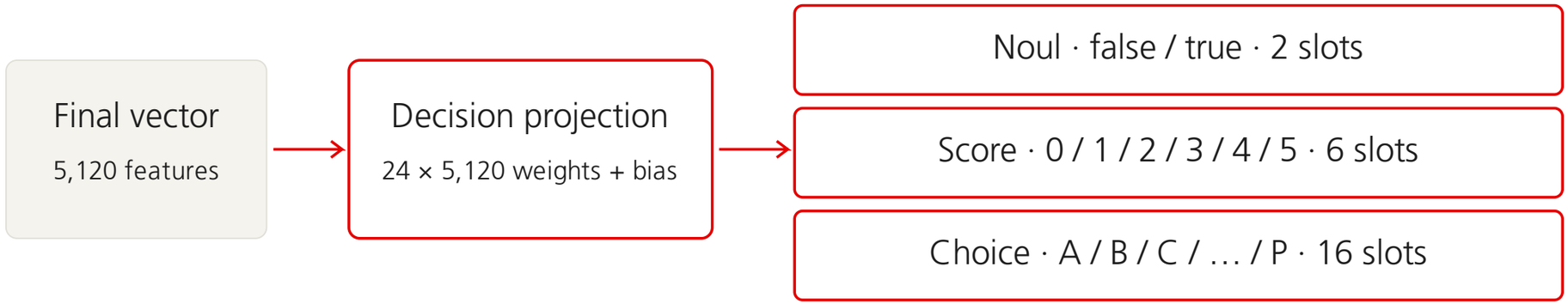
LoRA represents each update using two smaller matrices. Their low rank limits the number of trainable parameters. AutoTrust uses rank 16 and about 108 million LoRA parameters, around 0.4% of the backbone size.

These updates change how the model processes the evidence while retaining its pretrained weights. This is more powerful than adjusting the final probabilities alone: we are adapting the internal representation that will drive the decision.

Sources (checked 2026-10-23): Hu et al. · LoRA, 2021 · AutoTrust · adapter config · AutoTrust · pinned model card

Give the Model a Decision Ending

After the decoder stack, read the final position’s 5,120 features.
Train a small projection to decision slots instead of the whole vocabulary.



For our four options

A → Business B → Inheritance C → Employment D → Gift

Keep the active slots → divide by temperature → softmax.

The prompt defines what each slot means. The head learns how to score it.

AutoTrust · slot config · AutoTrust · pinned model card · AutoTrust · pinned implementation

Speaker notes

The final token position has 5,120 features. Instead of projecting those features onto the entire vocabulary, AutoTrust trains a small projection onto 24 decision slots, plus a bias.

The slots are organized by question type. A four-way Choice uses four of the sixteen available Choice slots. The prompt explains what A, B, C and D mean for this request. Tomorrow, those same slots can represent different categories.

This separates a stable output shape from runtime-defined meanings. The head is small, but it must be trained to interpret the final representation correctly. Temperature then adjusts the active slot distribution.

Sources (checked 2026-10-23): AutoTrust · slot config · AutoTrust · pinned model card · AutoTrust · pinned implementation

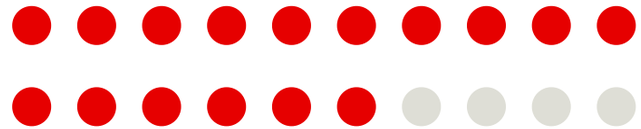
Reinforcement Learning for Calibrated Decisions

Learn which decisions to make and how much probability they deserve.

80%

Assigned probability

Across many decisions



About 16 in 20 outcomes occur

The stated objective

Train decisions with calibrated uncertainty.
Rewarding a good answer alone does not establish that its probability is reliable.

What remains private

The reward formula, training recipe, corpus contents and number of examples.

TypeSafe · AI primer · TypeSafe · launch, 15 Sep 2026

Next: AutoTrust's published approach learns from full teacher distributions.

Speaker notes

TypeSafe calls its approach reinforcement learning for calibrated decisions. The stated goal is to learn useful decisions together with probabilities that reflect their uncertainty.

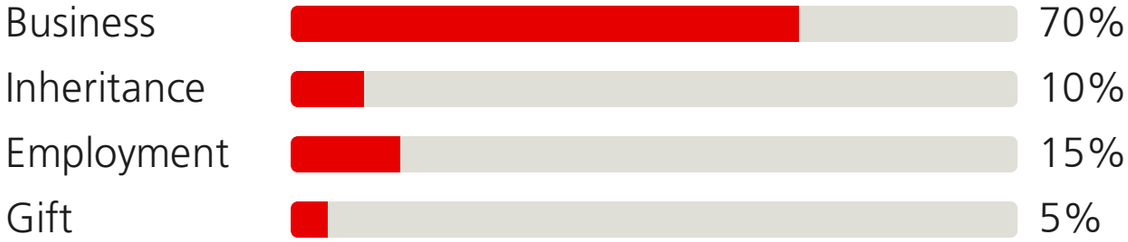
The picture illustrates the target: across many comparable predictions assigned 80%, about sixteen out of twenty should occur. It is a property of a collection of predictions, not a guarantee for an individual case.

TypeSafe has not published the reward formula, detailed training recipe or corpus size. We should not fill in those gaps. Standard language-model training already learns probabilities; the distinction here is a training objective directed at calibrated decisions. AutoTrust offers a different, published recipe.

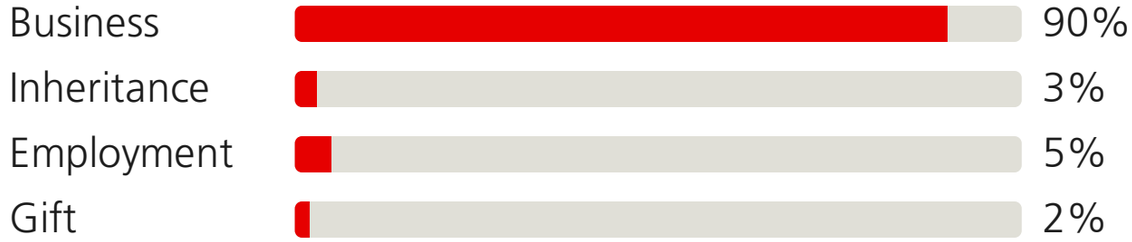
Sources (checked 2026-10-23): TypeSafe · AI primer · TypeSafe · launch, 15 Sep 2026

Train the Distribution, Not Just the Winner

Teacher target



Student prediction



Both pick Business. Training still penalizes the student's overconfidence.

1 · Learn

Train LoRA + slot weights + bias to match the teacher's full distribution.

2 · Calibrate

Freeze the trained model.
Fit a temperature per question type on held-out rows.

3 · Evaluate

Check decision accuracy and calibration against independent outcome labels.

AutoTrust: KL divergence over active slots; an extra ordered-score loss for Score.
Teacher-distribution distillation is its published recipe. TypeSafe describes its own training as RLCD.

AutoTrust · pinned model card · AutoTrust · fitted temperatures · TypeSafe · launch, 15 Sep 2026

Speaker notes

Both distributions choose Business, but they disagree about uncertainty. The teacher assigns 70%; the student assigns 90%. Training only the winning category would miss that difference.

AutoTrust trains its LoRA updates, slot weights and bias to match the full teacher distribution. It uses a distribution-matching loss, with an additional ordered-score loss for Score questions. After training, it fits temperatures on held-out rows.

Finally, evaluate against independent outcome labels. Matching a teacher is not proof of correctness or calibration: the student can inherit the teacher's mistakes. This is distillation from Jev probabilities, not TypeSafe's private RLCD recipe.

Sources (checked 2026-10-23): AutoTrust · pinned model card · AutoTrust · fitted temperatures · TypeSafe · launch, 15 Sep 2026

Keep the Knowledge. Learn the Odds.

LoRA
Small weight updates



Decision head
Slots + learned bias

Temperature
Fit after training

Small additions, trained on decisions and Jev's probabilities.

AutoTrust · pinned model card · AutoTrust · fitted temperatures

Speaker notes

Pause and simplify. The large sphere is the pretrained Qwen model. We keep that foundation and train relatively small components around it: internal LoRA updates, a decision head with bias, and temperature adjustments.

The result is a model adapted to decisions and teacher distributions. We have moved from reading arbitrary label-token scores to a purpose-trained readout. Now we need to see how it performs.

Sources (checked 2026-10-23): AutoTrust · pinned model card · AutoTrust · fitted temperatures

Three Models. The Same Benchmark Suite.

137 benchmarks · 26,269 judgments per model

Baseline-adjusted scores, 0–100 · higher is better

Model	Noul	Choice	Score	Task avg.
TypeSafe Jev 1.13	64.63	67.22	46.92	59.59
GPT-6 Luna Decisions API	54.54	56.23	36.85	49.21
AutoTrust JEV-27B	67.65	67.73	54.98	63.45

Task average gives equal weight to Noul, Choice and Score.

These are normalized task scores, not accuracy percentages.

Published S1MB runs · complete coverage · Luna's 17 English refusals count as errors.

S1MB · pinned results · S1MB · scoring

Speaker notes

This table compares three decision interfaces on the S1MB English suite. It covers 137 benchmarks and 26,269 judgments per model. Read the rows in order: TypeSafe, GPT-6 Luna Decisions, then AutoTrust.

These are baseline-adjusted scores, not accuracy percentages. The task average gives equal weight to Noul, Choice and Score. AutoTrust leads this aggregate snapshot; that does not mean it wins every task.

The table measures task performance, not overall calibration, latency or cost. GPT-6 Luna here is the native Decisions API, not a chat model generating JSON or a frontier reasoning run.

Sources (checked 2026-10-23): S1MB · pinned results · S1MB · scoring

A Closer Look at Shared Choice Tasks

Accuracy on four datasets · higher is better

Benchmark	TypeSafe Jev 1.13	GPT-6 Luna Decisions API	AutoTrust JEV-27B
BANKING77 Banking intent · 100 decisions	77.0%	69.0%	73.0%
MASSIVE 100 cases · 200 decisions	88.5%	84.0%	89.0%
CLINC Intent · 151 decisions	91.4%	82.1%	88.7%
GSM8K Four-choice math · 100 decisions	85.0%	69.0%	81.0%

Same recorded inputs across all three models. Fixed subsets of the full suite.

Runs: TypeSafe 28 Sep · GPT-14 Oct · AutoTrust 10 Oct 2026.

Speaker notes

These four examples make the aggregate easier to interpret. The first three involve intent classification; GSM8K is adapted into a four-choice math task. Here the figures are ordinary accuracy on the listed subsets.

The recorded inputs match across the three models. Notice that the ordering changes by task: AutoTrust leads MASSIVE, while TypeSafe leads the other three shown. Small differences on small subsets should not drive a universal ranking.

The engineering lesson is to compare on your own decision types, then look at the errors. An overall benchmark average cannot tell you which model is best for a particular workflow.

Sources (checked 2026-10-23): S1MB · pinned results

What Does Reasoning Buy?

Two medical Choice benchmarks · accuracy and recorded serving cost

Task / model	Accuracy	p50	p95	USD / 1K
PubMedQA · 500 research abstracts				
Jev 1.13	78.4%	0.27 s	0.35 s	$0.031
GPT-6 Sol · none	76.6%	1.19 s	1.62 s	$1.34
GPT-6 Sol · medium	78.2%	1.68 s	3.84 s	$1.85
DiagnosisArena-MCQ · 915 diagnostic cases				
Jev 1.13	59.8%	0.28 s	0.37 s	$0.032
GPT-6 Sol · none	79.2%	1.32 s	1.82 s	$1.46
GPT-6 Sol · medium	82.4%	2.39 s	5.14 s	$2.75

On complex cases, even Sol without reasoning has a large lead.

None / medium = reasoning effort. Jev: OpenRouter, sequential. Sol: OpenAI, 4 concurrent.
Client-side timings; Jev billed cost, Sol estimated from tokens. One run per condition.

Madrid-García & Merino-Barbancho · Tables 2 & S7

Speaker notes

This is a separate study using the same questions for Jev and GPT-6 Sol. None and medium refer to Sol’s reasoning effort. The columns show accuracy, median and tail latency, and cost per thousand items.

On PubMedQA, Jev and medium-reasoning Sol have similar accuracy. On difficult diagnostic cases, Sol has a large advantage even without reasoning. So the difference cannot be explained by reasoning tokens alone.

The recorded costs and timings are useful context, but serving conditions differ: Jev used OpenRouter sequentially; Sol used the OpenAI API with four concurrent requests. Do not treat the timing ratio as a controlled model-speed comparison.

Sources (checked 2026-10-23): Madrid-García & Merino-Barbancho · Tables 2 & S7

Jev Decides. Code Routes. Specialists Work.

1 · JEV CLASSIFIES

What is the source?

Transaction + supporting documents

Business

Inheritance · Employment · Gift

→

2 · CODE ROUTES

Choose the specialist

Business	→	**Business agent**
Inheritance	→	Estate agent
Employment	→	Income agent
Gift	→	Gift agent

→

3 · SPECIALIST ANALYZES

Work through the case

Use domain tools and, when needed, a reasoning model.

Return structured findings.

Jev checks each result field against the original evidence

Fast, low-cost checks as results arrive.

Amount supported	Payer supported	Shareholding history uncertain

→

Expert review

Code flags uncertain or unsupported fields immediately.

TypeSafe · intent routing · TypeSafe · extraction cascade

Speaker notes

Return to our transaction. Jev decides the source category. Code uses that result to route the material to a specialist. That specialist can use domain tools and a reasoning model where deeper analysis is valuable.

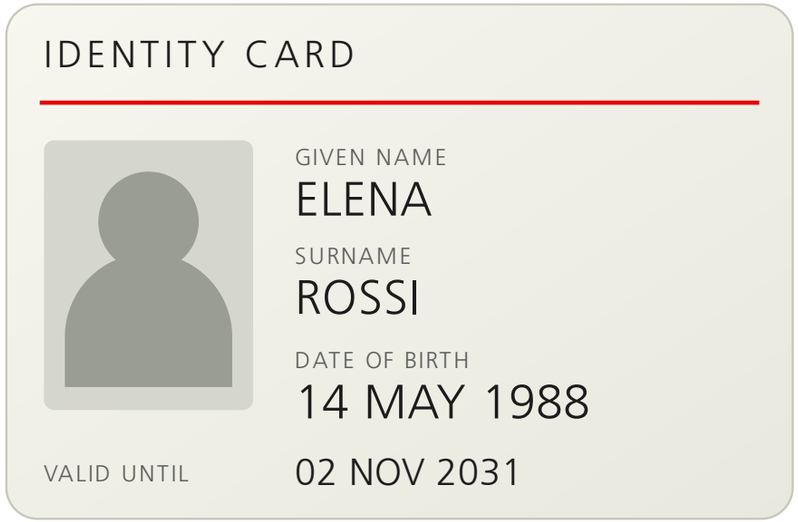
As structured findings arrive, Jev can check each field against the original evidence. The amount and payer may be supported, while the shareholding history remains uncertain. Code applies the review policy and surfaces the uncertain field to an expert.

The model supplies judgments; code controls the workflow. Validate thresholds and end-to-end latency on the actual workload before deciding how much review to automate.

Sources (checked 2026-10-23): TypeSafe · intent routing · TypeSafe · extraction cascade

Extract by Choosing

1 · Read the ID



OCR preserves text and nearby labels.

2 · Collect candidate spans

A Given name
ELENA

B Surname
ROSSI

C Date of birth
14 MAY 1988

D Valid until
02 NOV 2031

3 · Jev matches the fields

Given name
A → ELENA

Surname
B → ROSSI

Date of birth
C → 14 MAY 1988

Expiry date
D → 02 NOV 2031

One Choice per field.
Include “not found” when no span fits.

Jev selects the span. Code copies the text and validates the dates.

Speaker notes

Start with OCR: it turns the ID image into text spans and preserves nearby labels. Jev receives those candidate spans and chooses which one belongs to each requested field.

For given name, choose A; for surname, B; for date of birth, C. Code then copies the selected text rather than asking the model to generate it again. Code also parses and validates dates.

Include a not-found option when no candidate fits. Choosing a span prevents inventing a value outside the candidates, but it cannot repair a bad OCR result automatically. The candidate quality and the matching quality are separate checks.

Sources (checked 2026-10-23): TypeSafe · extract by choosing

THE TAKEAWAY

Give Intelligence the Right Interface

Keep the model's knowledge.

Ask for the decision you need.

Jev decides.

Calibrated probabilities.

Code executes.

Rules, routing and validation.

Reasoning goes deeper.

When the case needs it.

Where does your system generate text just to make a choice?

Let's discuss.

Sources ↗

Speaker notes

The takeaway is to give intelligence the right interface. Keep the model’s knowledge, ask for the decision you need, and use code to execute clear rules. Bring in deeper reasoning when the case calls for it.

Leave the audience with the question on screen: where does your system generate text just to make a choice?

Pause and invite discussion for the remaining five minutes. Useful prompts: which decision could you test first, what would an error cost, and what evidence would persuade you to automate it? The source appendix is linked here.

Sources (checked 2026-10-23): Presenter’s conceptual explanation or illustrative example; not measured model output.

Sources & Further Reading

TypeSafe Jev

Launch & System One ↗
API & typed decisions ↗
Models & pricing ↗
Parallel questions ↗
Speculative fan-out ↗
RLCD & machine-learning primer ↗

Architecture & calibration

The Illustrated GPT-2 ↗
Prefill & decode · NVIDIA ↗
Brown et al. · few-shot learning ↗
Guo et al. · calibration ↗
Hu et al. · LoRA ↗
Zhao et al. · prompt calibration ↗

Open implementations & benchmarks

AutoTrust JEV-27B · model card ↗
AutoTrust · implementation ↗
AutoTrust · fitted temperatures ↗
SemIf · calibration experiment ↗
S1MB · benchmark results ↗
S1MB · scoring methodology ↗

Practice & perspective

Intent routing ↗
Extraction & verification cascade ↗
Extract by choosing ↗
Raschka · Jev & generalization ↗
OpenAI · API pricing ↗
Jev vs reasoning · medical benchmarks ↗

Implementation and benchmark links use the revisions cited in the talk.

← Back to discussion

Speaker notes

Use this appendix during questions or after the talk. The links cover TypeSafe’s interface, the architecture explanations, calibration research, pinned open implementations, benchmarks and practical workflows.

Return to the closing slide for discussion. The source snapshot is dated so version-sensitive claims can be checked later.

Sources (checked 2026-10-23): Presenter’s conceptual explanation or illustrative example; not measured model output.